

Problem Statement:

Create a complete Agentic AI application called **ChefGPT AI – Intelligent Recipe Generator & Cooking Assistant** using **Python, Flask, IBM watsonx.ai Studio, IBM Granite Models, and a Document Q&A RAG system**.

Generate the entire project in a single file named app.py.

Do not use databases.

Do not create multiple files.

The primary goal is to demonstrate:

- IBM watsonx.ai Studio
- IBM Granite Models
- Retrieval-Augmented Generation (RAG)
- Agentic AI Architecture
- Personalized Recipe Generation
- Conversational Recipe Q&A

The project should be suitable for IBM SkillsBuild, hackathons, academic projects, and AI showcases.

Project Objective

Build an intelligent Recipe Assistant that helps users:

- Upload recipe documents
- Search recipes using natural language
- Adapt recipes based on dietary needs
- Generate personalized cooking instructions
- Create shopping lists
- Provide nutritional insights

All reasoning should be powered by IBM Granite Models.

Document Q&A RAG System

Implement a lightweight RAG pipeline.

Users can upload:

- Cookbooks
- Recipe PDFs
- Recipe Notes
- Cooking Guides
- Food Blogs (TXT format)

Supported Formats:

- PDF
- TXT

The system should:

1. Extract text from uploaded documents
2. Split content into chunks
3. Create embeddings
4. Store chunks in memory
5. Retrieve relevant chunks for user queries
6. Provide retrieved context to IBM Granite Models

This RAG layer should be the primary knowledge source.

Agentic AI Architecture

Implement five specialized AI agents.

Agent 1: Recipe Retrieval Agent

Purpose:

Retrieve the most relevant recipe information from uploaded documents.

Responsibilities:

- Search recipe knowledge base
- Retrieve matching recipes

- Identify relevant cooking instructions

Example Queries:

- Chocolate cake recipe
- Pasta recipes with mushrooms
- High-protein breakfast ideas

Generate:

- Retrieved Recipe Context
- Relevant Ingredients
- Cooking Notes

Use RAG + IBM Granite Models.

Agent 2: Recipe Adaptation Agent

Purpose:

Customize recipes based on user constraints.

Adapt recipes for:

- Vegetarian
- Vegan
- Gluten-Free
- Sugar-Free
- Keto
- High-Protein
- Low-Calorie

Input:

- Retrieved recipe
- User preferences

Generate:

- Modified Ingredients
- Updated Instructions

- Alternative Options

Use IBM Granite Models.

Agent 3: Ingredient Substitution Agent

Purpose:

Suggest ingredient replacements.

Example:

Input:

No Eggs Available

Output:

- Yogurt
- Flaxseed Mix
- Banana

Generate:

- Substitute Ingredient
- Reasoning
- Quantity Conversion

Use IBM Granite Models.

Agent 4: Nutrition Analysis Agent

Purpose:

Analyze recipe nutrition.

Generate:

- Calories
- Protein
- Carbohydrates
- Fat
- Fiber

Provide:

- Nutritional Summary
- Health Recommendations

Use IBM Granite Models.

Agent 5: Shopping List Agent

Purpose:

Generate shopping lists.

Responsibilities:

- Extract ingredients
- Organize by category
- Estimate quantities

Categories:

- Vegetables
- Dairy
- Proteins
- Spices
- Pantry Items

Generate:

- Smart Shopping List

Use IBM Granite Models.

Master Orchestrator Agent

Create a central Orchestrator Agent.

Responsibilities:

- Receive user queries
- Trigger RAG retrieval
- Route information to specialized agents

- Combine outputs into a final cooking guide

Implement:

- `recipe_retrieval_agent()`
- `recipe_adaptation_agent()`
- `substitution_agent()`
- `nutrition_agent()`
- `shopping_list_agent()`
- `orchestrator_agent()`

The orchestrator should act as the brain of the system.

IBM watsonx.ai Integration

Use IBM Granite Models for all reasoning and content generation.

Read credentials from:

WATSONX_API_KEY

WATSONX_PROJECT_ID

WATSONX_URL

Create reusable helper functions:

- `generate_response()`
- `retrieve_recipe_context()`
- `adapt_recipe()`
- `analyze_nutrition()`
- `generate_shopping_list()`
- `orchestrate_agents()`

Clearly comment all IBM watsonx.ai integration points.

Conversational Recipe Q&A

Users should be able to ask:

- How can I make a sugar-free chocolate cake?

- What can replace butter in brownies?
- Suggest a vegan pasta recipe.
- What can I cook in under 20 minutes?
- Give me a high-protein breakfast recipe.

The AI should answer using retrieved recipe knowledge.

Recipe Intelligence Dashboard

Create a professional dashboard.

Display:

Retrieved Recipe

- Recipe Name
- Source Context
- Matching Score

Personalized Recipe

- Modified Ingredients
- Updated Instructions

Nutritional Facts

- Calories
- Protein
- Carbs
- Fat

Ingredient Substitutions

- Suggested Replacements
- Reasoning

Shopping List

- Categorized Ingredients
 - Quantities
-

Agent Visualization Panel

This is critical.

Display:

Agent 1:

Recipe Retrieval Agent

Agent 2:

Recipe Adaptation Agent

Agent 3:

Ingredient Substitution Agent

Agent 4:

Nutrition Analysis Agent

Agent 5:

Shopping List Agent

Show:

- Agent Outputs
- Workflow Progress
- Why each agent was activated

This should clearly demonstrate Agentic AI behavior.

Flask Interface

Create a modern single-page application.

Users should be able to:

- Upload recipe documents
- Ask cooking questions
- Request recipe adaptations
- Generate shopping lists
- View nutritional analysis

Display all agent outputs visually.

User Interface

Use Bootstrap 5.

Design Requirements:

- Modern cooking-themed interface
- Responsive layout
- Recipe cards
- Nutrition panels
- Shopping list cards
- Agent workflow visualization

Use `render_template_string()`.

Do not create separate HTML files.

Deliverable

Generate a complete runnable `app.py` file.

The application must clearly demonstrate:

- IBM watsonx.ai Studio
- IBM Granite Models
- RAG-based Recipe Retrieval
- Agentic AI
- Multi-Agent Collaboration
- Personalized Recipe Adaptation
- Nutritional Analysis
- Shopping List Generation
- Conversational Recipe Q&A

The code should be beginner-friendly, heavily commented, visually appealing, and suitable for IBM SkillsBuild, watsonx.ai Studio demonstrations, hackathons, and academic presentations.